

Centrale Casablanca

ECC - Spring 2026

Introduction to AI and Machine Learning

Mini-Project

When Machine Learning Fails

Diagnosing and Repairing Failure Modes of a Non-Linear Model

Course	Introduction to AI and Machine Learning
Instructors	Kawtar Zerhouni and Rym Nassih - UTER MID@S
Nature of the project	Research-oriented investigation. Complements Lab 1 and Lab 2.
Team size	2 to 3 students
Deliverables	Code repository and written report
Grading	Graded out of 20 points

1. Why This Project Exists

In real-world systems, machine learning models do not fail randomly. They fail because a data distribution has shifted, because the problem was ill-posed from the start, because the inductive bias of the model is wrong for the data, or because a system around the model was poorly engineered. Models that reach 95 percent accuracy on a benchmark and 40 percent accuracy once deployed in production are not rare. They are the norm.

This project is deliberately different from Lab 1 and Lab 2. Those labs ask you to build a working pipeline and to deploy it under constraints. This project asks you to do something that most ML courses never require:

force a model to fail, understand why it fails, and repair it.

The posture is scientific rather than purely engineering. You will state a research question, formulate a falsifiable hypothesis about how a model breaks, design a controlled experiment to test that hypothesis, propose a correction targeting the cause rather than the symptom, and discuss the threats to the validity of your conclusions. This is the mindset that separates an engineer who ships models from an engineer who can debug them when they break.

2. What You Must Do

In order, your team must:

1. Pick one dataset from the authorized list in Section 4 (or propose another one, subject to instructor approval);
2. Train a working non-linear model on this dataset and establish a reference performance;
3. Formulate one explicit research question about a specific failure mode;
4. Design and run a controlled experiment that makes this failure mode visible and measurable;
5. Analyze the failure along the symptom / cause / test / correction structure described in Section 6;
6. Propose and implement a correction that targets the cause, not the symptom;
7. Evaluate the correction and discuss the threats to the validity of your conclusions;
8. Optionally, study a second failure mode more briefly (bonus points).

3. A Hard Constraint on the Model Family

You are not allowed to use linear or logistic regression in this project. This is a deliberate pedagogical choice.

On a linear model, the coefficients tell the story. When the model misbehaves, you can often read the problem directly from the weights. Real-world systems rely on ensembles and neural networks, where this direct interpretability is lost. Debugging such models requires a completely different skill set:

constructing diagnostic experiments, measuring the right quantities, and reasoning about invariances. This project is designed to build exactly that skill.

You must therefore train at least one of the following families:

- Tree-based ensembles (Random Forest, Gradient Boosting, XGBoost, LightGBM);
- Feed-forward neural networks (Multi-Layer Perceptron);
- Convolutional or recurrent architectures if they are justified by the data modality.

Linear or logistic regression may only be used as a sanity-check baseline at the very beginning; they cannot be the object of the failure analysis.

4. Authorized Datasets

You must choose one of the four datasets below. Each was selected because it admits several of the failure modes described in Section 5, so that the investigation remains meaningful. If you wish to propose a different dataset, you must obtain instructor approval during the first session.

Dataset	Task	Typical failure modes
SECOM	Binary classification of defective semiconductor chips from 590 sensor measurements.	Severe class imbalance, high dimensionality, missing values, spurious correlations.
Paddy (UCI 1186)	Regression or classification on agronomic and environmental features of rice production.	Mixed feature types, encoding sensitivity, distribution shift across seasons, small sample size.
Online Shoppers (UCI 468)	Binary classification of web sessions leading to a purchase.	Class imbalance, temporal drift (month effect), shortcut learning on traffic source.
MetroPT-3 (UCI 791)	Anomaly or failure prediction from industrial compressor sensor time series.	Strong temporal structure, rare failure events, non-stationarity, data leakage through naive splits.

A team that worked on one of these datasets in Lab 1 or Lab 2 may reuse it here, but must choose a failure mode that was not addressed before. A team that simply re-runs its Lab 2 pipeline under a new name will lose the 'originality' part of the grade.

5. Catalogue of Failure Modes

You must investigate one failure mode in depth. A second failure mode, treated more briefly, gives bonus points. The five failure modes below are authorized. You may propose another one subject to instructor approval.

5.1 Class imbalance failure

The model collapses toward the majority class, achieves high accuracy and near-zero recall on the minority class, and the business-relevant errors become invisible.

Typical symptoms: high overall accuracy, recall close to zero on the rare class, the confusion matrix is almost diagonal-majority.

Typical corrections: cost-sensitive losses, class weighting, SMOTE or other resampling, decision threshold optimization.

5.2 Overfitting and generalization gap

The model memorizes the training set and fails to generalize. Training performance is high, validation and test performance collapse.

Typical symptoms: large gap between training and validation curves, instability of performance across folds of cross-validation.

Typical corrections: regularization, pruning, early stopping, dropout, data augmentation, reduction of the hypothesis space.

5.3 Distribution shift

The distribution of the data changes between training and deployment. The model becomes progressively wrong without any visible warning, because the training pipeline keeps reporting nominal performance.

Typical symptoms: degradation of performance on a subset of the data that shares a specific attribute (time period, sensor, user segment), while aggregate metrics remain acceptable.

Typical corrections: feature redesign to be shift-robust, importance reweighting, periodic retraining, drift detection.

5.4 Feature representation failure

The raw features are not aligned with the quantity to be predicted. Even a powerful model cannot recover the signal because the information is not accessible in the current representation.

Typical symptoms: performance plateau that does not improve with model complexity, high variance across seeds, poor feature importances.

Typical corrections: feature engineering, domain-specific transformations, aggregation over time windows, dimensionality reduction.

5.5 Shortcut learning and spurious correlations

The model achieves very high performance by exploiting a feature that is correlated with the target in the training data but is not causally related to it. When that correlation breaks, the model collapses. The canonical example in the literature is the husky-versus-wolf classifier that had in fact learned to detect snow in the background.

Typical symptoms: excellent in-distribution performance, catastrophic failure on a slightly shifted test set, feature importances pointing to an implausible feature.

Typical corrections: removal or masking of the offending feature, evaluation on a stratified or adversarial test split, explicit invariance constraints.

A failure mode based purely on adversarial robustness (input-space perturbations, Lipschitz bounds) is not accepted in this project. The tooling required for a rigorous treatment is beyond the scope of the course.

6. Required Structure of the Analysis

The core of the project is the investigation of the chosen failure mode. This investigation must follow the four-step structure below. A report that does not explicitly separate these four steps will lose points on rigor.

6.1 Research question

State, in one or two sentences, the question you are investigating. The question must be precise enough to admit a yes-or-no answer after the experiments.

Example: *"Does the Random Forest trained on the Online Shoppers dataset rely on the Month feature as a shortcut, such that removing this feature significantly degrades in-distribution accuracy but improves performance on sessions from unseen months?"*

6.2 Observed symptom

Describe, with numbers and figures, what is actually observed on the model. State exactly which metric fails, on which subset of the data, and to what extent. An opinion is not a symptom. A confusion matrix, a learning curve, or a performance decomposition per subgroup is a symptom.

6.3 Causal hypothesis and controlled test

Formulate a hypothesis about the cause of the symptom. The hypothesis must be falsifiable: you must be able to state, in advance, an experimental result that would contradict it.

Then design and run a controlled experiment that isolates the hypothesized cause. Examples:

- To test a shortcut hypothesis, remove or permute the suspected shortcut feature and measure the drop in performance.
- To test a distribution shift hypothesis, split the data along the suspected axis (time, sensor, segment) and compare in-distribution and out-of-distribution performance.
- To test an imbalance hypothesis, artificially vary the imbalance ratio and measure how recall on the rare class degrades.

The experiment must include a proper control: a setting in which the cause is absent. Without a control, an experiment demonstrates nothing.

6.4 Correction that targets the cause

Propose a correction and justify why it attacks the cause identified in the previous step, rather than merely masking the symptom. Implement it, re-evaluate, and compare to the reference performance.

A correction that improves the main metric while leaving the cause in place (for example, tuning the decision threshold without addressing a distribution shift) is acceptable only if it is explicitly declared as such and discussed.

7. Threats to Validity

In the report, a dedicated subsection must discuss why the conclusions of the investigation might be wrong. This is standard scientific practice and is mandatory here. At minimum, the following questions must be addressed:

- Could the observed gain be explained by random variability across seeds rather than by the proposed correction?
- Is the test set large enough for the measured differences to be statistically meaningful?
- Could the correction be working for reasons unrelated to the identified cause (confounding effect)?
- Does the experimental protocol generalize beyond the specific dataset used?
- Is there any data leakage that would invalidate the comparison between the broken and the corrected model?

A team that acknowledges a limitation of its own analysis will be graded higher than a team that claims a perfect result.

8. Deliverables

8.1 Code Repository

Delivered as a single Git repository containing a README, a requirements file, one or several notebooks, and a fixed random seed. The broken and the corrected pipelines must both be runnable independently, so that the comparison can be reproduced by the instructors.

8.2 Report

The report must be structured exactly as follows:

9. Research question and chosen dataset;
10. Reference model and observed symptom;
11. Causal hypothesis and controlled experiment;
12. Proposed correction and evaluation;
13. Threats to validity;
14. Conclusion and what you learned.

Figures and tables are expected wherever they clarify the argument. There is no strict page limit: what matters is that each section of the report actually says something. Filler and repetition will be noticed and penalized on the 'clarity' criterion; conciseness will be rewarded.

9. Grading Rubric (out of 20)

The rubric reflects the priority given to the scientific quality of the investigation. Raw predictive performance is not directly graded. A team whose corrected model still fails, but whose analysis of that failure is rigorous, will be graded higher than a team with a better-performing model but a shallow explanation.

Criterion	Points	What is evaluated
Research question and setup	2	The question is precise, falsifiable, and worth investigating.
Symptom characterization	3	The failure is made visible and measured with appropriate diagnostics.
Causal hypothesis and controlled test	5	The hypothesis is falsifiable, the experiment includes a proper control, the conclusion is supported.
Quality of the correction	3	The correction targets the cause, not the symptom, and is justified.
Experimental validation	3	Before-and-after comparison is rigorous, metrics are appropriate, variance is reported.
Threats to validity	2	Self-critical discussion of why the conclusions might be wrong.
Clarity of the report	2	Structure, figures, quality of explanations, conciseness.
Bonus - second failure mode	+2	A second failure mode investigated briefly with the same four-step structure.
Total (capped at 20)	20	

10. What Makes a Strong Project

Four criteria separate a strong project from a merely correct one.

- **You forced the model to fail.** You did not wait for a failure to happen; you engineered a setting that makes it visible. This is the single most important skill in operational ML.
- **You explained why, not just what.** A confusion matrix is a symptom. An explanation is a causal account supported by a controlled experiment.

- **Your fix is not trivial.** Applying SMOTE because the classes are imbalanced is a trivial fix. Identifying that the imbalance hides a sampling bias in how the data was collected, and proposing a reweighting scheme based on that bias, is a non-trivial fix.
 - **You reasoned like a researcher, not only like an engineer.** You stated what you did not know, what you assumed, and what you could not test. You did not claim a perfect result.
-

In production, models do not fail because they are wrong.

They fail because someone did not ask why they could be wrong.